Evolutionary Computation: practical work

Computational Intelligence, Master in Artificial Intelligence (CI-MAI)
Lluís A. Belanche

Generated: 21/10/2026 19:52:58 (UTC+02:00 CEST)

Contents

1 Instructions **1**
1.1 Mandatory information 1
1.2 What we expect: a small scientific study 2
1.3 Suggestions 2

2 Training neural networks with evolutionary algorithms **2**

3 Evolution strategies: which mutation for which landscape? **3**

4 Genetic algorithms: how large must the population be? **3**

5 The travelling salesman problem: what should crossover preserve? **4**

6 Constraint handling: why does each method fail? **4**

7 Genetic programming: parsimony, bloat and generalisation **5**

8 The iterated prisoner's dilemma: when does cooperation emerge? **6**

9 Schema theory and deception **6**

1 Instructions

This document presents several practice problems in genetic algorithms (GAs), evolution strategies (ESs) and genetic programming (GP). Each group chooses **one** of them.

1.1 Mandatory information

- Deliver a brief PDF document (**10 pages maximum**, A4, including everything).
- Delivery date: no later than **6 November 2026**, via the Racó.
- Work in groups of **2 students** preferably (alternatively 3). Include **all names** in the PDF, and make only one delivery per group.
- Include a plain text file named `README.txt` with complete instructions on how to set everything up and how to obtain your results.
- If you use ChatGPT or a similar tool, tell us so, and say for what. We want to evaluate your work, not someone else's. See Citing generative AI for how to cite it.

1.2 What we expect: a small scientific study

Each exercise is built around a **research question**. The aim is not to run as many configurations as possible, but to answer that question convincingly with the smallest experiment that can do it. Your report should read like a short paper:

1. **Question and hypothesis.** State the question, and your hypothesis *before* running the experiments (you may propose your own). Say which result would refute it.
2. **Design.** Manipulate *one* factor (as in the lab: one knob), keep everything else fixed, and justify the fixed values. Include a baseline or control, i.e. a configuration whose outcome you can predict.
3. **Results.** Run enough independent runs to separate the effect from the run-to-run variability, and justify the number (a pilot of 10 runs shows how large the spread is). Report effect sizes with confidence intervals, not only p-values: for instance, the difference of medians with a bootstrap interval, or the Vargha–Delaney $\hat{A}_{12}$ statistic [23, 1]. For the tests, use the Mann–Whitney (Wilcoxon rank-sum) test for independent runs; the Wilcoxon signed-rank test only for paired runs (same seeds, same instances).
4. **Mechanism.** Explain *why* the result is what it is, and support the explanation with a measurement (a diversity curve, a count of feasible individuals, a heritability coefficient…), not only with the final fitness.
5. **Limitations.** What does your experiment *not* show? How far can the conclusion be generalised?

Make your results reproducible: fix the random seeds and document all parameters.

The "suggested defaults" of each exercise are starting points, not obligations. Each exercise also proposes possible extensions: a deeper analysis will be valued, but a well-answered core question is better than a superficial treatment of both.

1.3 Suggestions

- Please limit the choice of programming language to R, Julia or Python.
- The recommended packages are those of the lab: in R, `GA` (genetic algorithms, with your own operators) and `cmaes` (CMA-ES); in Python, `deap` (genetic algorithms and genetic programming, with your own operators) and `cma` (CMA-ES, by N. Hansen). The lab notebooks also contain short implementations of a GA, a (1+1)-ES and a self-adaptive $(\mu/\mu, \lambda)$-ES that you can reuse.
- R Markdown (`https://bookdown.org/yihui/rmarkdown/`) or Jupyter notebooks are good tools to keep code, results and text together.
- Keep the problem simple at first; once the whole thing works, you can make it more complex. Run a small pilot before the real experiment.

2 Training neural networks with evolutionary algorithms

The application of evolutionary search procedures to artificial neural networks is a long-standing research topic [25]. We can distinguish the evolution of the connection weights, of the architecture (number of layers, neurons per layer, connectivity), of the hyper-parameters, of the activation functions and of the learning rules. Evolutionary algorithms bring clear advantages over derivative-based methods: the error function need not be continuous or differentiable, it can incorporate discrete information (such as the number of neurons), and it can be noisy. On the other hand, the error landscape is *deceptive* (similar networks may perform quite differently) and *multimodal* (exchanging hidden neurons together with their weights does not change the network) [25], and the computational cost is high.

Research question. With the same computational budget, does an evolutionary algorithm find networks that *generalise* better than a derivative-based method, or only networks that fit the training data as well? Does the answer depend on the size of the training sample?

A possible hypothesis: since the EA does not follow the gradient into the nearest sharp minimum, its advantage (if any) appears with small or noisy training samples, and disappears with large ones.

Core. Use synthetic data from a generator that you control (the precise problem, classification or regression, is up to you): then the true generalisation error of a network can be approximated by its error on a large test set, and the best achievable error by that of the noise-free generator. Use a multilayer perceptron with one hidden layer of fixed size, and evolve its weights with a GA or an ES. Compare it with a derivative-based method (backpropagation, BFGS…) at the *same number of error-function evaluations or the same time*,

stating which one you equalise and why. The single factor is the training-set size (e.g. 3–4 values); keep the validation and test sets fixed and large.

Possible extensions. Evolve also the architecture (number of hidden neurons) or the regularisation (weight decay), with a validation set as fitness. Does the EA choose a model size that tracks the training-set size in a sensible way?

Suggested defaults (change them if you have a reason, and say why). input dimension 2–5, 5–20 hidden neurons, weights in $[-5, 5]$, test set of 5000 points.

Implementation hints. *R:* `GA`, `cmaes`, `nnet` (derivative-based baseline); time with `system.time()` or the package `tictoc`.

Python: `deap`, `cma`, `scikit-learn` (`MLPClassifier`, `MLPRegressor`) or `PyTorch` as the derivative-based baseline; time with `time.perf_counter()`.

3 Evolution strategies: which mutation for which landscape?

The minimisation of multivariate functions is a long-standing problem in mathematics; classical approaches use first- and second-order derivatives [4]. Evolution strategies need none, and adapt the mutation distribution instead [3]: one step size σ, n step sizes $\sigma_1, \ldots, \sigma_n$, or a full covariance matrix (correlated mutations; nowadays, CMA-ES [12]). The lab showed one example of each. Here you study *when* each representation pays off.

Research question. How does the advantage of n step sizes over one step size depend on the conditioning of the problem, and what happens to it when the problem is no longer separable?

A possible hypothesis: n step sizes help more and more as the conditioning grows, but only if the problem is separable (its principal axes are the coordinate axes); on a rotated problem the advantage disappears, and only correlated mutations (CMA-ES) recover it.

Core. Use a family of problems with a single hardness knob, for instance the ellipsoid $f(\mathbf{x}) = \sum_{i=1}^{n} \kappa^{(i-1)/(n-1)} x_i^2$ with condition number $\kappa \in \{1, 10^2, 10^4, 10^6\}$. For each κ, measure the number of evaluations needed to reach a target value (e.g. $f < 10^{-8}$) with a self-adaptive ES with one and with n step sizes, and its rotated version $f(Q\mathbf{x})$ (Q a random orthogonal matrix). Runs that do not reach the target within the budget are *censored*: report how many, and do not average them as if they had succeeded. Plot evaluations against κ on log–log axes: what is the slope of each curve? Support the explanation by plotting the evolution of the step sizes.

Possible extensions. Add CMA-ES to the comparison, and/or a second knob: the dimension n (how do the curves scale with n?) or additive noise in f (which representation is more robust?).

Suggested defaults (change them if you have a reason, and say why). $n = 10$, $(\mu/\mu, \lambda)$-ES with $\mu = 3$, $\lambda = 12$, learning rates of the lab, budget 10^5 evaluations, start at a random point in $[-5, 5]^n$.

Implementation hints. *R:* the ES of the lab notebook (Part 2) and `cmaes`.

Python: the ES of the lab notebook (Part 2) and `cma`.

4 Genetic algorithms: how large must the population be?

Many combinatorial problems have a natural binary representation, and integer variables can be coded with binary or Gray codes [10]. Classical exact approaches are branch-and-bound, branch-and-cut and dynamic programming [18]. A GA has a very practical question to answer before anything else: the size of its population. Too small, and the building blocks are lost by chance before selection can favour them; too large, and evaluations are wasted. The *gambler's ruin* model [13] predicts that, for a problem made of m independent blocks of k bits, the population needed to solve all the blocks with a given reliability grows roughly as $2^k \sqrt{m}\, \ln m$ (the factor $\ln m$ appears because every one of the m blocks must be solved).

Research question. How does the minimal population size that solves a problem reliably scale with the problem size, and does the scaling agree with the gambler's ruin model?

Core. Use concatenated trap functions [9]: for a block $\mathbf{b}$ of k bits with $u(\mathbf{b})$ ones, $f_{\text{trap}}(\mathbf{b}) = k$ if $u(\mathbf{b}) = k$,

and $k-1-u(\mathbf{b})$ otherwise; the fitness of a string of m blocks is the sum over the blocks, with the optimum at the all-ones string. Fix k (e.g. $k=4$), vary the number of blocks m (e.g. $m \in \{5, 10, 20, 40\}$), and for each m find, by bisection on the population size, the smallest population for which, say, 90% of the runs find the optimum. Plot $\mu_{\min}$ against m on log–log axes and compare it with the prediction $\sqrt{m}\ln m$ (whose local slope between $m=5$ and $m=40$ is about 0.9). If they disagree, find out why (hint: one-point crossover also disrupts blocks). Use OneMax ($k=1$) as the control. Explain the mechanism by tracking, for each block, the proportion of the population that holds it solved.

Possible extensions. Repeat for a second value of k (how does $\mu_{\min}$ change with k?), or study what happens if the bits of each block are scattered over the string instead of contiguous (the role of the defining length under one-point crossover, and the advantage of uniform crossover, if any).

Suggested defaults (change them if you have a reason, and say why). tournament selection ($k_t = 2$), one-point crossover with $p_c = 0.9$, bit-flip mutation with $p_m = 1/(mk)$ or none, stop when the population has converged.

Implementation hints. *R:* `GA` (`type = "binary"`; your own operators via the arguments `selection`, `crossover`, `mutation`) or the GA of the lab notebook.

Python: `deap` (`tools.cxOnePoint`, `tools.cxUniform`, `tools.mutFlipBit`, `tools.selTournament`) or the GA of the lab notebook.

5 The travelling salesman problem: what should crossover preserve?

The travelling salesman problem (TSP) is probably the most studied problem in combinatorial optimisation (see `https://www.math.uwaterloo.ca/tsp/`). Representing a tour as a permutation forces special crossover and mutation operators [16]. Some of them (partially mapped crossover, PMX; cycle crossover, CX) try to preserve the *positions* of the cities in the parents, others (order crossover, OX; edge recombination, ER [24]) their relative *order* or their *adjacencies*. But the length of a tour depends only on its edges.

Research question. Do crossover operators that pass on the edges of the parents work better than those that pass on positions, and is the heritability of edges the reason?

Core. Use `ulysses22` from TSPLIB (Homer's *Odyssey* [11]; optimum 7013, with the geographical distances of TSPLIB). Compare two or three crossover operators, all else equal. Define the *edge heritability* of an operator as the fraction of the edges of a child that appear in at least one of its parents, measure it, and relate it to the performance of each operator.

Possible extensions. Repeat on a larger instance of TSPLIB (100 cities or more): does the ranking of the operators hold? Or add a local search (2-opt) to the GA: does it change the ranking?

Suggested defaults (change them if you have a reason, and say why). population 100, tournament selection, inversion mutation, elitism of 1–2 tours, 1000 generations.

Implementation hints. *R:* `GA` with `type = "permutation"` (crossovers `gaperm_oxCrossover`, `gaperm_pmxCrossover`, `gaperm_cxCrossover`; mutations `gaperm_simMutation`, ...); `TSP` reads TSPLIB files.

Python: `deap` (`tools.cxOrdered`, `tools.cxPartialyMatched`, `tools.mutShuffleIndexes`); `tsplib95` reads TSPLIB files.

6 Constraint handling: why does each method fail?

Many problems are subject to constraints [6]:

$$\begin{aligned} \text{minimise} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & g_j(\mathbf{x}) \le 0, \quad j = 1, \dots, m, \\ & h_k(\mathbf{x}) = 0, \quad k = 1, \dots, p, \\ & \mathbf{x}^L \le \mathbf{x} \le \mathbf{x}^U. \end{aligned}$$

The constraint violation is $\phi(\mathbf{x}) = \sum_{j=1}^{m} \max(0, g_j(\mathbf{x})) + \sum_{k=1}^{p} \max(0, |h_k(\mathbf{x})| - \theta)$, with a tolerance θ for the equalities (usually $\theta = 10^{-4}$). Three families of methods:

- **static penalty**: minimise $F(\mathbf{x}) = f(\mathbf{x}) + r\,\phi(\mathbf{x})$ with a fixed r;
- **feasibility rules** [7]: feasible beats infeasible; between two feasible, the smaller f wins; between two infeasible, the smaller ϕ wins;
- **stochastic ranking** [20] or the **ε-constrained method** [21]: compromises between the two previous ones.

Two test problems of the CEC 2006 benchmark [5] behave very differently:

- **G09**: 7 variables in $[-10, 10]$, 4 nonlinear inequality constraints (2 active at the optimum); the feasible region is about 0.5% of the search space; $f^* = 680.630057$;
- **G11**: minimise $x_1^2 + (x_2 - 1)^2$ subject to the **equality** $x_2 - x_1^2 = 0$, with $x_1, x_2 \in [-1, 1]$; the feasible region has measure zero, so the tolerance θ is essential; $f^* = 0.7499$ (for $\theta = 10^{-4}$).

Research question. How do the size and the shape of the feasible region determine which constraint-handling method succeeds, and why?

A possible hypothesis: a static penalty is sensitive to r because the optimum lies on the boundary of the feasible region (too small r: the population converges outside; too large: it never explores the boundary from the infeasible side); feasibility rules avoid the choice of r, but on an equality constraint they push the population onto a thin set where it loses diversity.

Core. Compare a static penalty (with two or three values of r) and the feasibility rules on G09 and G11. Besides the final f and the success rate, follow the *fraction of feasible individuals* and the violation ϕ of the population over the generations: they show the mechanism.

Possible extensions. Add a third method (stochastic ranking or ε-constrained) and test whether it fixes the failure you identified; or move to an engineering problem, such as the pressure vessel or the welded beam [6]. (For the pressure vessel, with the two thicknesses restricted to multiples of 0.0625 as in the original formulation, the best known value is $f^* \approx 6059.714$; with all variables continuous it is lower, about 5885.3, so state which version you solve.)

Suggested defaults (change them if you have a reason, and say why). population 100, 500 generations, simulated binary crossover (SBX, $\eta_c = 20$, $p_c = 0.9$), polynomial mutation ($\eta_m = 20$, $p_m = 1/n$) [8], tournament selection of size 2. Alternatively, an ES of the lab.

Implementation hints. *R:* `GA` has no SBX or polynomial mutation: write them (about ten lines each) and pass them as `crossover` and `mutation`.

Python: `deap` provides `tools.cxSimulatedBinaryBounded` and `tools.mutPolynomialBounded`; `scipy.stats.mannwhitneyu` for the tests.

7 Genetic programming: parsimony, bloat and generalisation

Genetic programming [15, 19] evolves programs, here mathematical expressions represented as trees, to fit observed data (*symbolic regression*). Internal nodes are functions (e.g. $+, -, \times, /, \sin, \cos, \log$; with protected division and logarithm, which return 1 and 0 when the argument is too close to zero) and leaves are variables and random constants. Trees tend to grow without improving the fit (*bloat*) [19]. A common remedy is *parsimony pressure*: maximise $F(T) = -\mathrm{MSE}(T) - \lambda\,|T|$, where $|T|$ is the number of nodes.

Research question. Does parsimony pressure only make trees smaller, or does it also make them generalise better? Is bloat what limits generalisation?

A possible hypothesis: a moderate λ reduces the size of the trees a lot at almost no cost in training error, and improves the test error when the data are noisy, because large trees fit the noise.

Core. Use two problems:

- **Nguyen-7** [22]: $f(x) = \log(x + 1) + \log(x^2 + 1)$, $x \in [0, 2]$, 20 noise-free training points;
- **Noisy rational**: $f(x) = (x^3 - x)/(x^2 + 1)$, $x \in [-3, 3]$, 30 training points with Gaussian noise of standard deviation 0.3.

For each, use an independent test set of 100 noise-free points. The single factor is λ (e.g. 0 and two or three values in $[10^{-3}, 10^{-2}]$). Track the size of the best tree and the training and test errors over the generations. Simplify the best expressions algebraically and comment on them.

Possible extensions. Make the noise level the second factor: does the best λ grow with the noise, as for a regularisation parameter? Or compare parsimony pressure with a hard limit on the depth of the trees.

Suggested defaults (change them if you have a reason, and say why). population 500, ramped half-and-half initialisation (depths 2–6), subtree crossover ($p_c = 0.9$), subtree mutation ($p_m = 0.1$), tournament selection of size 7, 100 generations.

Implementation hints. *R:* there is no maintained GP package: represent trees as nested lists or R expressions (build them with `call()` and evaluate them with `eval()`); `data.tree` helps to inspect them.

Python: `deap` has a GP module (`deap.gp`: primitive sets, `genHalfAndHalf`, `cxOnePoint`, `mutUniform`, `staticLimit`); `sympy` to simplify the expressions found.

8 The iterated prisoner's dilemma: when does cooperation emerge?

The iterated prisoner's dilemma (IPD) is the classical model of the emergence of cooperation [2]. Two players simultaneously cooperate (C) or defect (D), with payoffs

	Player 2: C	Player 2: D
Player 1: C	(R, R)	(S, T)
Player 1: D	(T, S)	(P, P)

$$T = 5,\ R = 3,\ P = 1,\ S = 0,$$

which satisfy $T > R > P > S$ (defection dominates in a single round) and $2R > T + S$ (mutual cooperation is better than alternating exploitation). In the iterated version the same two players play N_r consecutive rounds (e.g. 150), and each can condition its action on the history; the fitness of a strategy is its total payoff against the strategies it meets. Since fitness depends on the rest of the population, this is *coevolution*. Real players make mistakes: with *action noise*, each intended action is flipped with probability p_{noise}. Under noise, tit-for-tat (TFT) can get locked in long series of mutual retaliation, while win–stay, lose–shift (PAVLOV) corrects them [17].

Research question. Does cooperation emerge and remain stable in an evolving population, and how does the answer depend on the noise level?

A possible hypothesis: without noise, the population evolves towards TFT-like strategies; with moderate noise, towards more forgiving strategies (PAVLOV-like); with high noise, cooperation collapses.

Core. Represent a strategy as a lookup table with memory 1: one bit for the first action, plus one bit for each outcome CC, CD, DC, DD of the previous round (own action first), i.e. 5 bits. Then there are only 32 strategies, and you can name them (ALLD, TFT, PAVLOV, GRIM…) and follow their frequencies in the population. Evolve a single population in which every strategy plays against all the others (round robin), with $p_{\text{noise}} \in \{0, 0.01, 0.05\}$. Measure the average payoff per round (3 means full cooperation, 1 full defection) and the frequency of the main strategies over the generations. Check your explanation against the payoff matrix of the classical strategies under each noise level.

Possible extensions. Use memory 2 (2 bits for the first two actions plus 16 entries, 18 bits) or finite state machines with 4 states (an initial state of 2 bits, and for each state an output bit and two transitions of 2 bits: $2 + 4 \times 5 = 22$ bits). Do more complex strategies change the conclusion? Do you observe cycles (cooperation, invasion by defectors, recovery)?

Suggested defaults (change them if you have a reason, and say why). population 100, 500 generations, uniform crossover ($p_c = 0.7$), bit-flip mutation ($p_m = 0.01$ per bit), tournament selection of size 3, elitism of 2.

Implementation hints. *R:* actions as integers (0 = C, 1 = D); with 32 strategies, compute once the expected payoff matrix of all pairs; `parallel` for the games.

Python: the same data structures with `numpy`; `deap` for the evolutionary loop.

9 Schema theory and deception

Background (notation of the lecture). A *schema* $S \in \{0, 1, *\}^n$ is the set of strings that agree with S on its fixed positions; its *order* $\omega(S)$ is the number of fixed positions, and its *defining length* $\delta(S)$ is the distance

between the first and the last fixed positions. Under **fitness-proportional selection**, one-point crossover with probability p_c and bit-flip mutation with probability p_m,

$$\mathbb{E}\big[m(S,t+1) \mid P_t\big] \;\geq\; m(S,t)\,\frac{f(S,t)}{\bar{f}(t)}\Big(1 - p_c\,\frac{\delta(S)}{n-1}\Big)(1-p_m)^{\omega(S)},$$

where $m(S,t)$ is the number of instances of S in the population P_t, $f(S,t)$ their mean fitness and $\bar{f}(t)$ the mean fitness of P_t. The *building block hypothesis* states that GAs work by discovering and recombining short, low-order, high-fitness schemata [10]. A problem is *deceptive* when the low-order schemata lead away from the optimum. The **fully deceptive 3-bit function** [10] has its global optimum at 111:

string	000	001	010	011	100	101	110	111
f_3	28	26	22	0	14	0	0	30

but all order-1 schemata point to 000: $f(0{*}{*}) = 19 > f(1{*}{*}) = 11$, $f({*}0{*}) = 17 > f({*}1{*}) = 13$ and $f({*}{*}0) = 16 > f({*}{*}1) = 14$; the order-2 schemata behave in the same way.

Research question. How accurately does the schema theorem predict the dynamics of a real GA, and what can rescue a GA on a deceptive problem: a larger population, or crossover?

Core. Two parts.

- *Prediction.* On OneMax ($n = 30$), with fitness-proportional selection (roulette wheel or SUS: the theorem is stated for it), select about 10 schemata with different $\omega(S)$ and $\delta(S)$. At each generation, compare the actual count $m(S,t+1)$ with the bound computed from generation t. When is the bound tight, and when loose? Why?
- *Deception.* Concatenate 10 copies of f_3 (30 bits, maximum 300). With tournament selection, measure the success rate (all ten blocks at 111) as a function of the population size, with crossover ($p_c = 0.9$) and without it ($p_c = 0$). Use OneMax as the control. Track the proportion of solved blocks over the generations to explain what you see.

Possible extensions. A problem with two levels of deception (27 bits). Divide the string into three 9-bit super-blocks, each made of three 3-bit sub-blocks. At the first level, each sub-block is evaluated with f_3. At the second level, let $s_i = 1$ if all three sub-blocks of super-block i are 111 and $s_i = 0$ otherwise, and apply f_3 to the pattern $s_1 s_2 s_3$:

$$f(\mathbf{x}) = \sum_{j=1}^{9} f_3(\text{sub-block}_j) + \lambda\, f_3(s_1 s_2 s_3), \qquad \lambda = 10.$$

The unique global optimum is the all-ones string, $f = 270 + 300 = 570$. Solving one or two super-blocks is penalised at the second level, while solving none is rewarded: the best strings with no solved super-block reach $264 + 280 = 544$, whereas those with exactly two solved super-blocks reach only 268. Do the conclusions of the core still hold? Does the population size needed grow, and how much? Alternatively, measure the fitness–distance correlation [14] of the three problems and discuss whether it predicts their difficulty.

Suggested defaults (change them if you have a reason, and say why). population sizes from 50 to 1000, one-point crossover, bit-flip mutation with $p_m = 1/n$, run until the population converges.

Implementation hints. *R:* represent schemata as integer vectors over $\{0,1,2\}$ with $2 = *$; count the instances in a population matrix P with `sum(apply(P, 1, function(x) all(x == S | S == 2)))`; the selection functions of the lab notebook (roulette, SUS) can be reused.

Python: the same representation with `numpy`: `np.all((P == S) | (S == 2), axis=1).sum()`; the selection functions of the lab notebook can be reused.

References

[1] A. Arcuri, L. Briand. A practical guide for using statistical tests to assess randomized algorithms in software engineering. *Proc. ICSE*, 1–10, 2011.

[2] R. Axelrod. The evolution of strategies in the iterated prisoner's dilemma. In L. Davis (ed.), *Genetic Algorithms and Simulated Annealing*, Pitman, 1987.

[3] H.-G. Beyer, H.-P. Schwefel. Evolution strategies: a comprehensive introduction. *Natural Computing* 1(1):3–52, 2002.

[4] S. Boyd, L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. https://web.stanford.edu/~boyd/cvxbook/

[5] J. J. Liang et al. Problem definitions and evaluation criteria for the CEC 2006 special session on constrained real-parameter optimization. Technical report, Nanyang Technological University, 2006.

[6] C. A. Coello Coello. Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: a survey of the state of the art. *Computer Methods in Applied Mechanics and Engineering* 191(11–12):1245–1287, 2002.

[7] K. Deb. An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering* 186(2–4):311–338, 2000.

[8] K. Deb, R. B. Agrawal. Simulated binary crossover for continuous search space. *Complex Systems* 9(2):115–148, 1995.

[9] K. Deb, D. E. Goldberg. Analyzing deception in trap functions. *Foundations of Genetic Algorithms 2*, 93–108, 1993.

[10] D. E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.

[11] M. Grötschel, M. Padberg. The optimized Odyssey. *AIROnews* 6(2):1–7, 2001.

[12] N. Hansen, A. Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation* 9(2):159–195, 2001.

[13] G. Harik, E. Cantú-Paz, D. E. Goldberg, B. L. Miller. The gambler's ruin problem, genetic algorithms, and the sizing of populations. *Evolutionary Computation* 7(3):231–253, 1999.

[14] T. Jones, S. Forrest. Fitness distance correlation as a measure of problem difficulty for genetic algorithms. *Proc. ICGA 6*, 184–192, 1995.

[15] J. R. Koza. *Genetic Programming*. MIT Press, 1992.

[16] P. Larrañaga, C. M. H. Kuijpers, R. H. Murga, I. Inza, S. Dizdarevic. Genetic algorithms for the travelling salesman problem: a review of representations and operators. *Artificial Intelligence Review* 13(2):129–170, 1999.

[17] M. Nowak, K. Sigmund. A strategy of win-stay, lose-shift that outperforms tit-for-tat in the Prisoner's Dilemma game. *Nature* 364:56–58, 1993.

[18] C. H. Papadimitriou, K. Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Prentice-Hall, 1982.

[19] R. Poli, W. B. Langdon, N. F. McPhee. *A Field Guide to Genetic Programming*. Lulu, 2008. http://www.gp-field-guide.org.uk

[20] T. P. Runarsson, X. Yao. Stochastic ranking for constrained evolutionary optimization. *IEEE Trans. Evolutionary Computation* 4(3):284–294, 2000.

[21] T. Takahama, S. Sakai. Constrained optimization by the ε constrained differential evolution with gradient-based mutation and feasible elites. *Proc. IEEE CEC*, 2006.

[22] N. Q. Uy, N. X. Hoai, M. O'Neill, R. I. McKay, E. Galván-López. Semantically-based crossover in genetic programming. *Genetic Programming and Evolvable Machines* 12(2):91–119, 2011.

[23] A. Vargha, H. D. Delaney. A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics* 25(2):101–132, 2000.

[24] D. Whitley, T. Starkweather, D. Fuquay. Scheduling problems and traveling salesmen: the genetic edge recombination operator. *Proc. ICGA 3*, 133–140, 1989.

[25] X. Yao. Evolving artificial neural networks. *Proceedings of the IEEE* 87(9):1423–1447, 1999.